

Modeling a City Space To Support Orientation Skills Of the Blind Users With a Dedicated Ontology And a Neighborhood Graph

1st Alicja Rojek

dept. Faculty of Exact and Natural Sciences (WNSP.)
University of Siedlce (UWS.)
Siedlce, Poland
ar85287@stud.uws.edu.pl

2nd Dariusz Mikulowski

dept. Faculty of Exact and Natural Sciences (WNSP.)
University of Siedlce (UWS.)
Siedlce, Poland
dariusz.mikulowski@uws.edu.pl

Abstract—Independent functioning of BVI (blind or visually impaired) people in the city involves significant stress and orientation challenges. The aim of this research was to develop a semantic model of urban space using a dedicated ontology and graph database. As part of the work, a model (60 classes, 431 individuals) describing infrastructure and neighborhood relations was prepared. A proprietary Java application with the Apache Jena library was used for data exploration, comparing the performance of SPARQL queries and the API. Neighborhood relations were exported to a Neo4j database, creating a graph with 235 nodes and 493 edges. Using this graph, several pathfinding algorithms were used to determine the best routes for potential navigation. Tests of these algorithms (including Dijkstra's and Yen's) demonstrated high efficiency, achieving response times below 32 ms. The experiment confirmed that the integration of ontologies with graphs provides an effective basis for navigation systems. A key conclusion is the necessity of introducing edge weights for route personalization—favoring safe sidewalks and avoiding complex intersections. This solution can support spatial orientation training in safe simulation conditions.

Index Terms—Ontology, Space orientation, Blind users, Semantic languages, Graph path finding

I. INTRODUCTION

Independent functioning in a complex urban environment is one of the greatest challenges for blind and visually impaired people. This process is often fraught with high levels of stress, especially in situations of disorientation or accidental entry onto the roadway in an unauthorized place.

Nowadays, technology is a huge help for navigation. While many people use basic GPS apps, blind users can use special tools like Dotwalker [1], Seeing Assistant Move [2], and Wayfinder [4]. These apps are specifically made to help people walk safely outside by providing more detailed help than standard maps. Additionally, there are new inventions like smart glasses that use sensors to find obstacles. These devices can describe the environment out loud, helping users understand exactly what is around them in real time. However, finding the way inside buildings is still a big problem. Systems like TotuPoint [3] and NaviSecure [7] exist, but learning how to use them and staying oriented is still difficult. Educational games by Mikulowski and Szuba [6] are a great way to

practice these skills. We want to use ontologies to help technology understand how things like sidewalks and crossings are connected. This helps us create safe ways to practice navigation in a laboratory.

The development of modern information technologies opens up new possibilities for creating intelligent systems supporting spatial orientation. The key aspect here is not only the mapping of geometric data, but above all the semantic capture of relations and dependencies between urban infrastructure elements, such as sidewalks, pedestrian crossings, or architectural obstacles. Creating a formalized knowledge model in the form of an ontology allows for the construction of advanced navigation systems and simulation environments that enable safe orientation skills training in laboratory conditions.

The structure of this paper is divided into five main parts. The Literature section provides an overview of semantic data representation and its importance for navigation systems for the visually impaired. In the Experiment Procedure, the development of the CityOntoNavi model and the methods for semantic exploration and graph construction are described in detail. The Results section analyzes the quantitative data and performance efficiency of the pathfinding algorithms. Finally, the Conclusions summarize the experimental findings and suggest future work for developing personalized navigation systems for blind individuals.

II. LITERATURE

Ontologies form the foundation of semantic data representation, enabling the unification of heterogeneous sources of information about urban space. The literature points to their key role in building interoperable geographic information systems [8]. These technologies find particular application in systems dedicated to people with visual impairments. For example, the works of Mikulowski [5] and Mikulowski and Szuba [6] explore the use of ontologies in virtual sound reality (Virtual Sound Reality – VSR) environments supported by binaural sound, which allows for the creation of immersive educational scenarios. Despite numerous studies on geographic ontologies, there is still a gap in the integration of semantic spatial models

with efficient graph-based pathfinding mechanisms that would be tailored to the specific needs of blind people. Exploration of such models requires the use of dedicated query languages, such as SPARQL, and programming libraries (e.g., Apache Jena), whose performance is crucial when processing large urban datasets.

III. EXPERIMENT PROCEDURE

As part of the conducted research, a proprietary application in Java was developed, using the Apache Jena library to manage the urban ontology saved in RDF/OWL format. The research process was divided into several stages:

- 1) Preparation of the ontology: A model comprising 60 classes in a hierarchical structure was constructed (including InfrastructureForPedestrians, RoadInfrastructure, ElementsOfCityArchitecture). 431 individuals (urban objects) and key relations were defined, such as hasDirectNeighbor (neighborhood), isLocatedOn (location of an object on another object), and recordedInTheLocation (assignment of sound messages to objects).
- 2) Semantic exploration: The application allowed for executing SPARQL queries and operations via the Jena API to search for neighborhood relations. Performance tests were conducted, which showed that while the first SPARQL query may take longer due to loading data into memory, this approach exhibits high efficiency with large datasets.
- 3) Graph construction and path analysis: Neighborhood relations were exported to CSV format and then imported into the Neo4j graph database. The created graph consisted of 235 nodes and 493 edges. Using the Cypher language and the Graph Data Science (GDS) library, shortest path algorithms were tested: ShortestPath, Dijkstra, and Yens.
- 4) Test scenario: An example task involved determining a path between the objects stairsBankZachodniWBK and cityHallUrzadMiastaSkwerNiepodleglosci. The algorithms returned a consistent route comprising 5 intermediate nodes (mainly sections of sidewalks and streets), and the operation time fluctuated between 28–32 ms.

A. Preparation of the Ontology (CityOntoNavi)

The first step of the study was to load a specialized digital map, called the CityOntoNavi ontology, into a custom-made computer program. This digital map is a detailed representation of a real part of the city of Siedlce. It includes much more than just a basic layout; it features specific urban items like streets, buildings, traffic lights, advertising poles, and even ATMs. To keep all this information organized, we created 60 different categories (known as classes) arranged in a "family tree" structure. This means large groups, such as "Road Infrastructure," contain smaller, more specific groups like "Infrastructure for Pedestrians". Inside these categories, we placed 431 individual objects found in the city. Each object is described by its own set of details, such as its name and

its exact location using geographic coordinates as shown in Fig. 1.

```
<owl:NamedIndividual rdf:about="http://www.semanticweb.org/ln/ontologies/2019/0/cityOntoNavi#stairBankZachodniWBK">
  <rdf:type rdf:resource="http://www.semanticweb.org/ln/ontologies/2019/0/cityOntoNavi#stairBankZachodniWBK">
  <CityOntoNavi:hasDirectNeighbor
    <rdf:resource="http://www.semanticweb.org/ln/ontologies/2019/0/cityOntoNavi#stairBankZachodniWBK">
  <CityOntoNavi:location_gps_coordinates>52.165811, 22.271035</CityOntoNavi:location_gps_coordinates>
  <rdf:comment>Zachodnia PKS.</rdf:comment>
</owl:NamedIndividual>
```

Fig. 1. Example triple from the ontology.

What makes this system truly useful is how these objects are linked together. We defined 32 different types of connections (properties) to show how things relate to one another, as illustrated in Fig. 2. For example:

- 1) hasDirectNeighbor: This tells the system which objects are right next to each other in the real world.
- 2) isLocatedOn: Specifies the exact street or infrastructure element where an object can be found.
- 3) recordedInTheLocation: This allows the system to attach specific sound messages to certain spots, which is essential for helping people navigate the city.

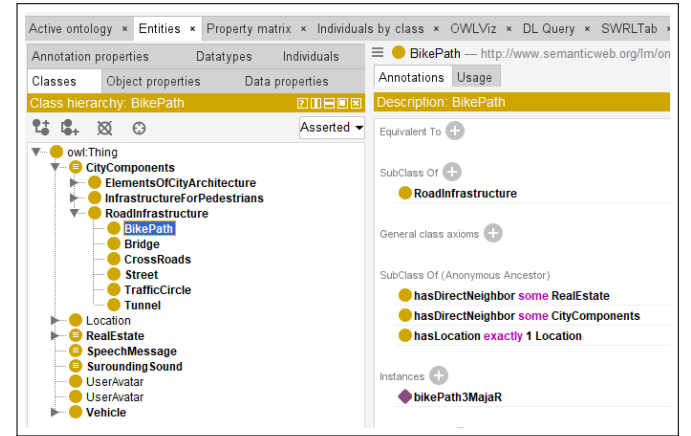


Fig. 2. A class hierarchy with annotations for a selected class in the Protege.

To manage all this data, we developed a Java application with a simple visual interface shown in Fig. 3.

Using this program, users can easily browse the city map, search for specific items, and perform basic operations such as adding, editing, and deleting objects, as well as defining new relationships between them (see Fig. 4).

However, it is important to note that the application does its work in the computer's temporary working memory (RAM). This means that any changes or new information must be manually saved to a file, or they will be lost once the program is turned off.

B. Semantic Exploration and Query Scenarios

The application we built makes it easy to find specific objects in the city by looking at the groups (classes) they belong to or the details (properties) they have, as illustrated in Fig. 5.

For example, a user can search for all "traffic lights" or find a specific "bank" based on its name, as well as search for

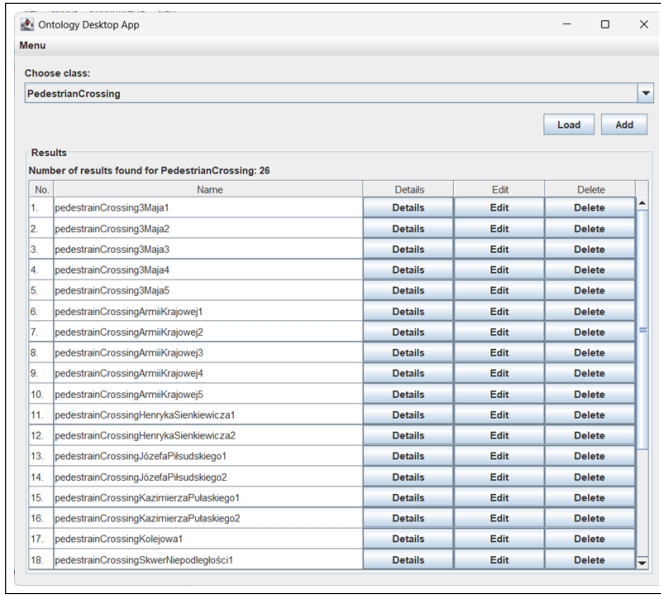


Fig. 3. A main screen of ontology exploration desktop application.

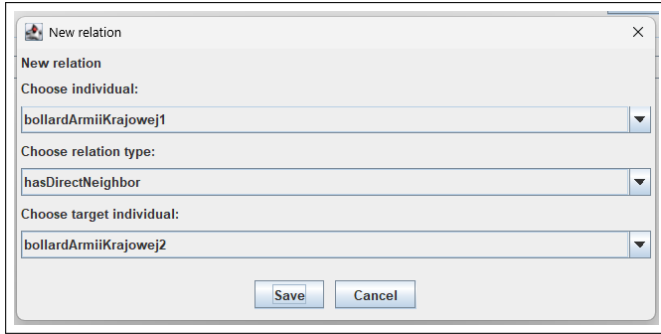


Fig. 4. Adding a new relation to the ontology.

relationships in the ontology that they have previously added (see Figs. 4 and 6). A major part of our test was comparing two different ways the computer can "think" when searching for this information:

- 1) The SPARQL Method (Declarative): This approach utilizes a high-level, declarative query language to define the specific criteria of the data to be retrieved. Instead of providing instructions on how to access the information, the user specifies the desired semantic relationships, and the system's query engine determines the most efficient way to extract the results from the urban ontology.
- 2) The Jena API Method (Imperative): This approach relies on the imperative execution of operations via the Apache Jena library. It requires the implementation of explicit logic to programmatically traverse the ontology's graph structure, following a specific sequence of manual instructions to identify and retrieve individuals and their associated properties.

During our tests, we noticed that the SPARQL method might take a few extra seconds the very first time it is used because the computer has to load all the city data into its active

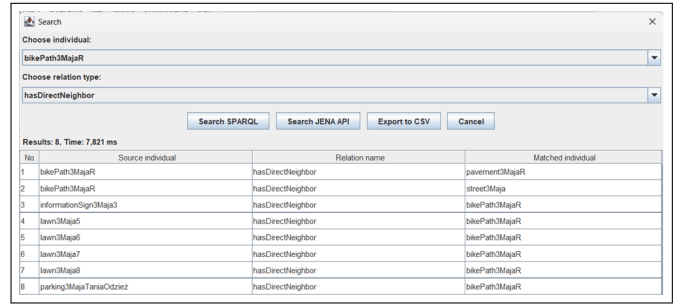


Fig. 5. Searching individuals and relations in the application.

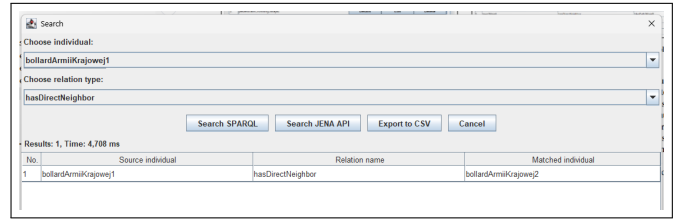


Fig. 6. Searching for recently added relations in the application.

memory. However, once the data is loaded, this method is very fast and powerful, even when there is a huge amount of information to look through. The most important finding was that both methods gave the exact same results. No matter which way the computer searched for objects connected by a specific relation, the answer was always identical. This proves that our system is reliable and that both ways of searching work correctly for navigating city data.

C. Graph Construction and Neo4j Integration

To do more advanced work with finding paths, we needed to move the information from the ontology into a format better suited for maps. We took the "neighborhood relations" (which show which city objects are next to each other) and exported them into a simple CSV file. The exported data was structured into semantic triples, where each record in the CSV file consisted of three distinct components: the source individual, the relational property (such as hasDirectNeighbor), and the target individual. Next, we brought this data into Neo4j, which is a database designed to handle maps and networks. We used a specific language called Cypher to tell the database how to read the CSV file and create the graph. To make sure the map was accurate, we set a "uniqueness rule" (called a constraint) for the names of the objects. This ensured that each city landmark appeared only once in the system, preventing any confusing duplicates. The final result was a complete, digital map of that part of the city. This network, or graph, was made up of 235 nodes (representing individual city objects like buildings or street corners) and 493 edges (the lines representing the connections between them). Because many objects share the same neighbors, the system automatically linked everything together into one large, connected structure, making it possible to calculate routes from one side of the area to the other.

D. Pathfinding Analysis and Test Scenario

The graph we built in the previous step was very important because it allowed us to look for the best ways to travel between different city objects. To do this, we used a special set of tools called the Graph Data Science (GDS) library, which is a part of the Neo4j system designed specifically for analyzing connections. We wanted to see how well the system could find a path in a real-world situation. For our test, we chose a specific task: finding a route from the stairs at Bank Zachodni WBK to the City Hall at Skwer Niepodległości. We did not just use one method; we tested three different mathematical methods, or algorithms, called ShortestPath, Dijkstra, and Yens, to see if they would all agree on the best way to go. The results were very successful. Every algorithm we tried found the exact same path. This route took the user through 5 different intermediate points as shown in Fig. 7, which were mostly segments of sidewalks and streets. What was even more impressive was the speed of the system. The computer was able to calculate the entire path in just 28 to 32 milliseconds. This experiment proves that taking detailed information from an ontology and putting it into a graph database is a very fast and effective way to handle city navigation tasks.

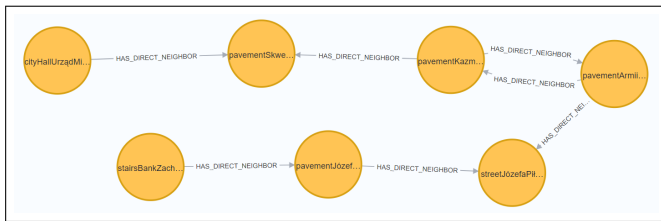


Fig. 7. Example of a graph representation of the computed shortest path.

IV. RESULTS

The experiment provided clear data and facts that prove our city navigation system works well. The digital map, called CityOntoNavi, was built with 60 different categories for things like roads and sidewalks. Inside these categories, we added 431 specific city objects, such as buildings and traffic lights. We also created 32 types of connections to link these objects together. Two very important links are `hasDirectNeighbor`, which shows which things are next to each other, and `isLocatedOn`, which tells the system exactly which street an object is on. We tested two different ways for the computer to search for information: the SPARQL method and the Jena API method. Our tests showed that both ways found the exact same answers, which proves the system is reliable. We also found that while the SPARQL method needs a moment to load the data at first, it is very fast at handling large amounts of city information once it starts. To find the best routes, we moved the city data into a system called Neo4j. This created a digital map with 235 points (nodes) and 493 connections (edges). We then asked the computer to find the shortest path between two specific places: the stairs at Bank Zachodni WBK and City Hall. Every method we used found the same route, which included 5 steps along various sidewalks and streets. The computer was extremely

fast, finishing the calculation in only 28 to 32 milliseconds. This proves that the experiment was a success. It shows that combining a detailed digital map with a fast graph database is an excellent way to help people find their way around a city quickly and accurately.

V. CONCLUSIONS

The conducted experiment confirmed the effectiveness of combining object ontologies with graph databases in modeling urban space. The study focused on developing a custom Java application to manage a digital map of Siedlce containing 431 objects, which was then integrated with a graph database to test the speed and accuracy of finding city routes. The main conclusions from the research are as follows: Route optimization: Routes determined by standard shortest path algorithms require further personalization for blind people. It is necessary to introduce graph edge weights that would favor continuous sections of sidewalks and the presence of landmarks (e.g., sound signals) while avoiding complex intersections. Technological performance: The integration of SPARQL and Neo4j allows for efficient processing of spatial relations, which constitutes a solid foundation for navigation engines in assistive systems. Limitations and the future: The current challenge remains the labor-intensiveness of manual ontology modeling. Future work should focus on automating data acquisition (e.g., through LiDAR or OCR) and on testing generated routes with blind users in a VR environment.

This method has shown great potential in building training systems that will allow people with visual impairments to safely familiarize themselves with the city's topography before going out into the field.

REFERENCES

- [1] Dotwalker User Guide, <https://dotwalker.prv.pl/dwusermanualwstep.html>, [Accessed 29-05-2026]
- [2] Tutorial Seeing Assistant Move, <https://seeingassistant.tt.com.pl/move/tutorial>, [Accessed 29-05-2026]
- [3] TOTUPOINT - System origins, <https://totupoint.pl/geneza>, [Accessed 29-05-2026]
- [4] Our Solution - Wayfinder App, <https://wayfinderapp.co.uk/services/>, [Accessed 29-05-2026]
- [5] Mikułowski, D. (2017). An Approach for Ontologically Supporting Space Orientation Teaching of Blind Pupils Using Virtual Sound Reality. International Conference on Interactive Collaborative Learning.v
- [6] Mikułowski, D., Szuba, B. (2025). Supporting the Development of Spatial Orientation Skills of Blind People Using Binaural Sounds in the VSR Game. Studia Informatica. System and information technology.
- [7] Mikułowski, D., Niewiadomski, A., Salamonczyk, A., Pilski, M., Switalski, P., Terlikowski, G. (2022, November). Supporting Independent Navigation Of Disabled Students In University Campus Using Beacons And Ontology Map. In EdMedia (pp. 1005-1010). Association for the Advancement of Computing in Education (AACE).
- [8] Sun, K., Zhu, Y., Pan, P., Hou, Z., Wang, D., Li, W., Song, J. (2019). Geospatial data ontology: the semantic foundation of geospatial data integration and sharing. Big Earth Data, 3, 269 - 296.